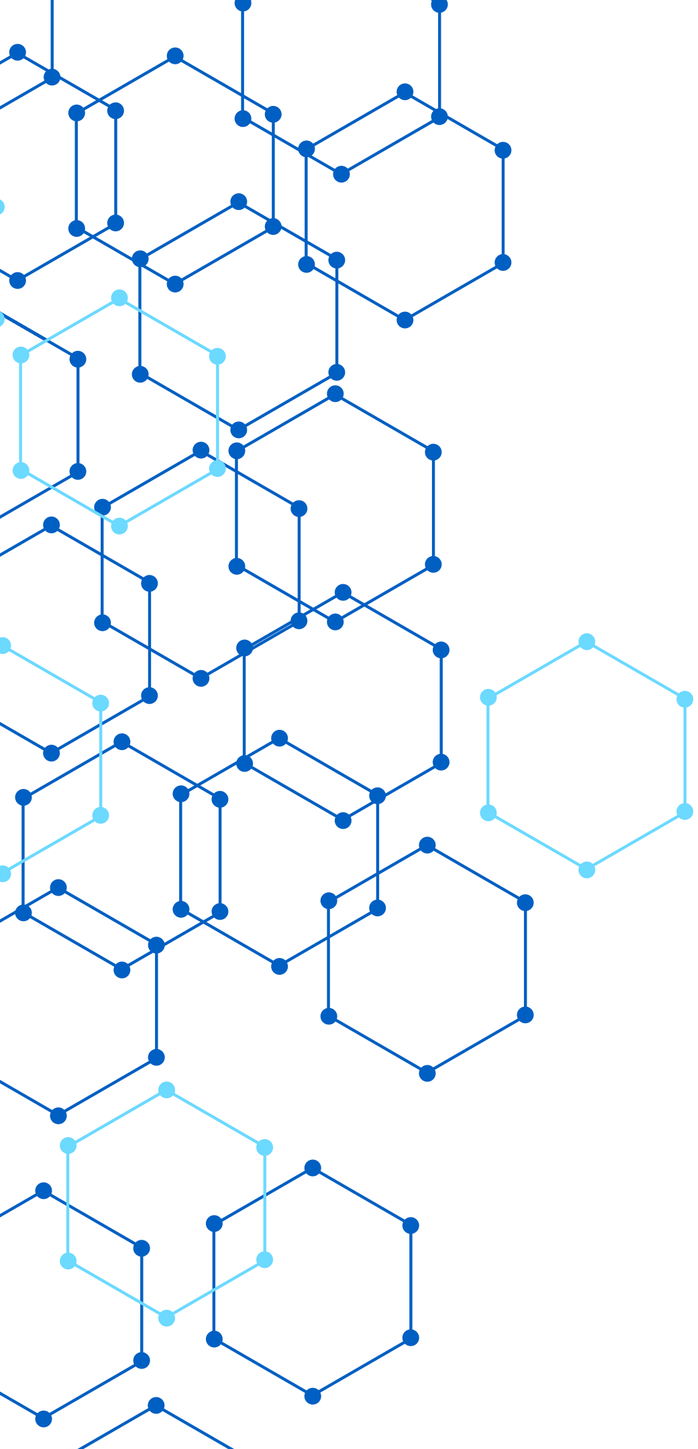
OBSERVABILIDAD EN SPRING BOOT

Prometheus · Loki · Grafana



¿QUÉ ES OBSERVABILIDAD?

Es la capacidad de entender el estado interno de un sistema a partir de las señales que emite, incluso ante preguntas que no anticipaste.

Viene de la teoría de control (Kalman, 1960): un sistema es observable si sus salidas permiten inferir su estado.

En software esas salidas son métricas, logs y trazas. La prueba de fuego: si para entender un problema nuevo tienes que agregar un log y redesplegar, te falta observabilidad.

MONITOREO VS. OBSERVABILIDAD

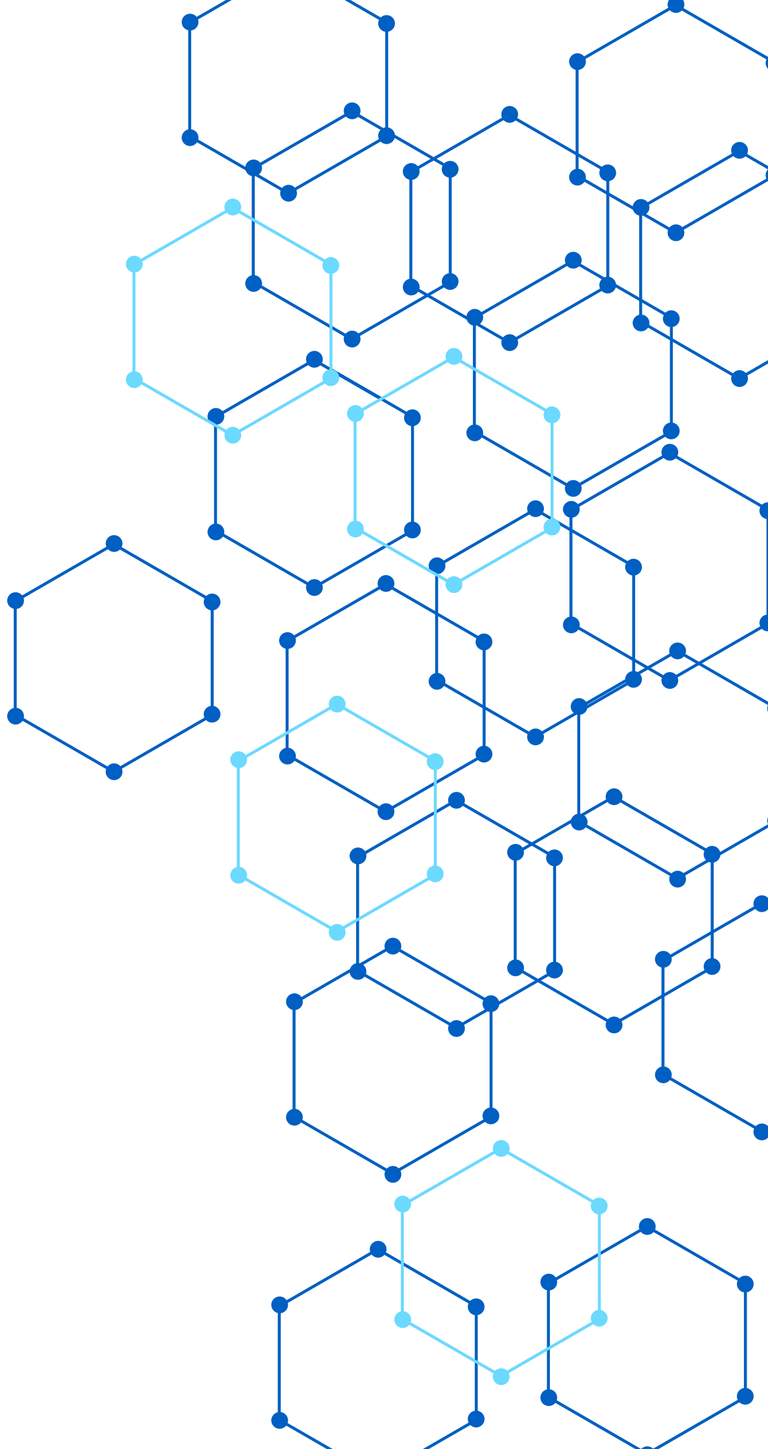
Monitoreo responde «¿está funcionando?». Vigila problemas que ya conoces con umbrales y alertas. Ejemplo: la CPU pasa del 80% y suena una alerta.

Observabilidad responde «¿por qué se comporta así?». Te deja investigar problemas nuevos con telemetría rica. Ejemplo: el p99 de /pagos sube solo cuando responde una pasarela.

¿CUÁNDO USARLA?

Sí, desde el día uno: microservicios, dependencias externas (pagos, APIs de terceros), SLOs que cumplir, varios equipos sobre el mismo sistema y despliegues frecuentes.

Con mesura: en un prototipo o un monolito pequeño, Actuator y logs estructurados ya alcanzan. La telemetría también cuesta (disco, cardinalidad), así que empieza por RED en los endpoints críticos.



LOS TRES PILARES

MÉTRICAS

Números agregados en el tiempo: cuánto, qué tan rápido, qué porcentaje. Baratas e ideales para alertar.

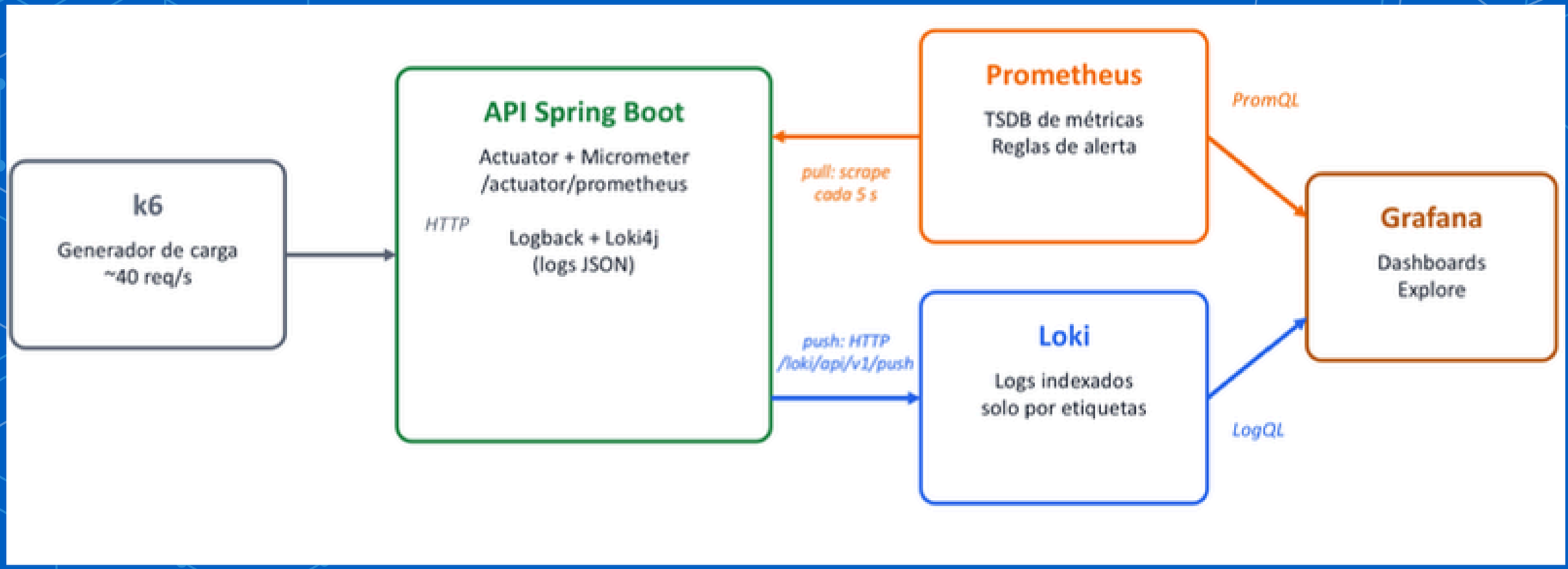
LOGS

Eventos con contexto: qué pasó en este request. Si son JSON, se consultan como datos.

TRAZAS

El recorrido de un request entre servicios: dónde se fue el tiempo. Clave en microservicios.

ARQUITECTURA DE LA DEMO

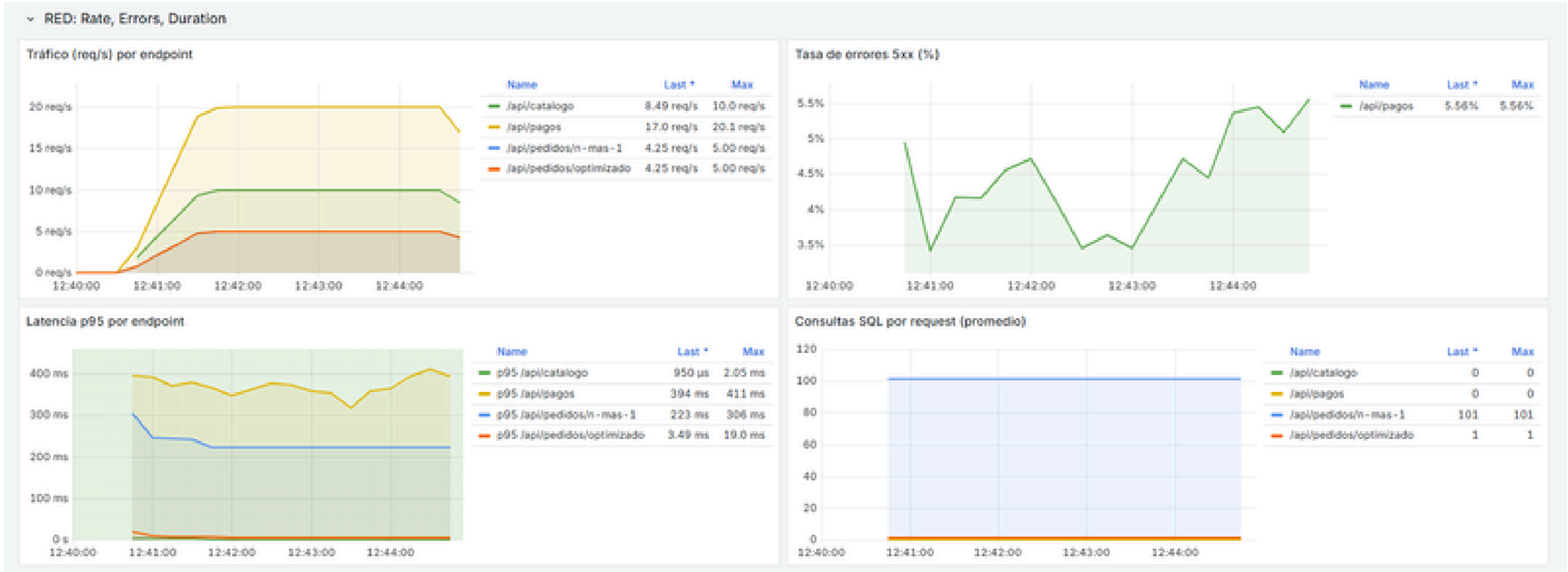


Prometheus va por las métricas cada 5 s (pull). Los logs JSON viajan a Loki (push). Grafana consulta ambos con PromQL y LogQL. Todo se levanta con docker compose.

LA DEMO: UNA TIENDA CON TRAMPAS

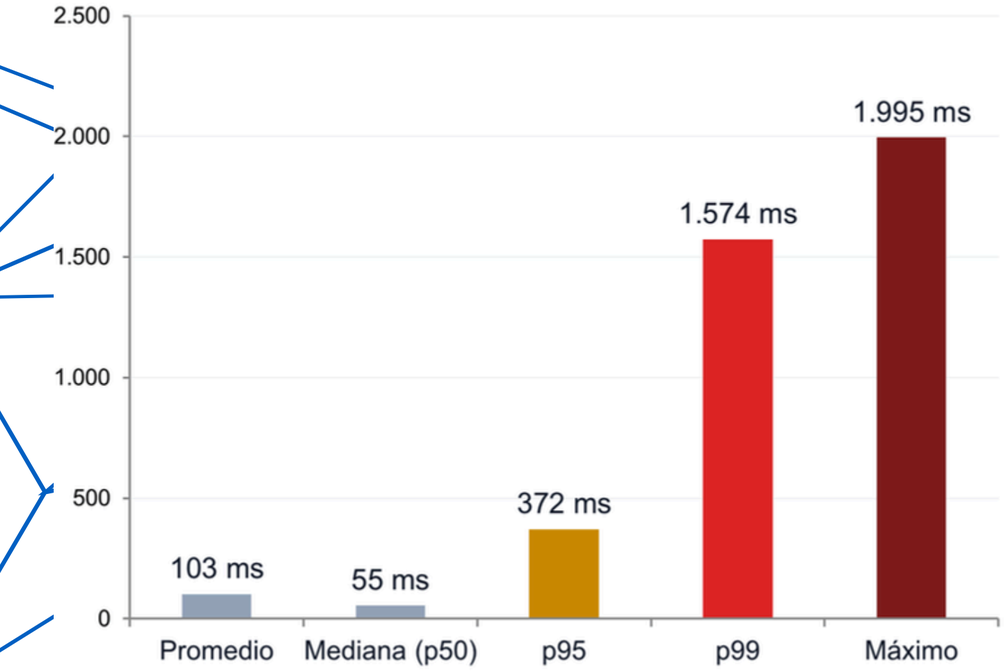
- POST /api/pagos llama a una pasarela externa que a veces es lenta y falla cerca del 4% de las veces (20 req/s).
- /api/pedidos/n-mas-1 y /optimizado devuelven los mismos 100 pedidos: uno con N+1 y otro con JOIN FETCH (5 req/s cada uno).
- k6 envió 9.603 requests en 4 minutos (unos 40 por segundo). /api/catalogo es el control sano (10 req/s).

EL DASHBOARD EN GRAFANA



Datos reales de la prueba de carga: solo /api/pagos tiene errores, su p95 ronda los 400 ms y /api/pedidos/n-mas-1 hace 101 consultas SQL por request.

CUELLO 1: EL PROMEDIO MIENTE



- En /api/pagos el promedio es 103 ms, pero el p99 llega a 1.574 ms: 1 de cada 100 pagos tarda más de 1,5 segundos.

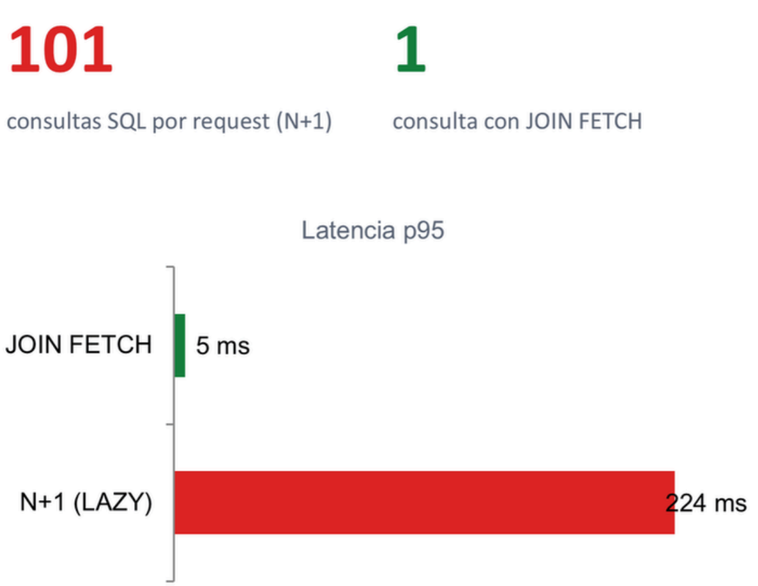
- La causa es la pasarela externa, no el servidor: máximo 5 de 50 hilos ocupados y CPU bajo 6%. k6 midió lo mismo desde el cliente (p99 de 1.559 ms).

CUELLO 2: EL PROBLEMA N+1

/api/pedidos/n-mas-1 hace 101 consultas SQL por request: 1 para los pedidos y 1 más por cada uno, porque la relación es LAZY.

Con JOIN FETCH es una sola consulta y el p95 baja de 224 ms a 5 ms: unas 45 veces más rápido.

Lo delató una métrica propia y el log JSON con mdc_sql_consultas = 101.



DE LA ALERTA AL LOG

- La alerta TasaErroresAlta se disparó: falló el 4,4% de los pagos (213 de 4.801) y el umbral era 2%.
- El dashboard acota el problema: solo /api/pagos tiene errores; pedidos y catálogo están en cero.
- Loki explica el porqué: «La pasarela de pagos respondio timeout», y cada línea trae su request_id.





RED Y USE: QUÉ MEDIR

- RED, para servicios: Rate, Errors, Duration. Es la vista del usuario. En la demo marcó /pagos (cola y errores) y /pedidos/n-mas-1 (lento).

- USE, para recursos: Utilization, Saturation, Errors. La JVM estaba sana: heap bajo 145 MiB, 5 de 50 hilos y CPU bajo 6%. Escalar servidores no lo habría arreglado.

BUENAS PRÁCTICAS

- Logs en JSON con contexto (request_id, uri, duración) y percentiles con histogramas: SLOs en p95 y p99, nunca en el promedio.
- Alerta por síntomas que ve el usuario (errores, latencia) e investiga por causas. Etiquetas de baja cardinalidad: /pedidos/{id}, nunca el id.
- Mide también el negocio (pagos aprobados y fallidos) y versiona dashboards, datasources y alertas en Git.





ERRORES COMUNES

CARDINALIDAD

userId o requestId como etiqueta: millones de series y Prometheus se cae. Lo mismo pasa con Loki.

RUIDO

Alertas sin acción clara que el equipo termina silenciando, y dashboards de 40 paneles que nadie mira.

CEGUERA

Confiar en el promedio (103 ms con un p99 de 1.574 ms) e instrumentar solo después del incidente.

TALLER: AHORA LO ENCUENTRAN USTEDES

- Minutos 0 a 6: abren un GitHub Codespace (o Docker local), corren docker compose up y lanzan la carga con k6.
- Minutos 6 a 16: tres misiones. La cola de latencia (PromQL), cazar el N+1 (LogQL) y de la alerta al request_id.
- Minutos 16 a 20: reto. Crear una alerta de p99 mayor a 1 s y verla dispararse. La guía paso a paso está en TALLER.md.

¿PREGUNTAS?

Gracias